
Final Project Report

CS-340



Intro to AI

Author: Ioannis Karantounias

E-Mail: 20240177@student.act.edu

Supervisor: Astaras Alexandros

Introduction

This project is meant to assist us in understanding the basics of AI model architecture (obviously not at the low level) as well as familiarize us with some more advanced concepts. Through this project and course concepts like sigmoid functions, epochs, topologies and more were not only introduced but explored and implemented providing for a clear learning experience. We got to turn data into numbers and explore how models react to tweaking constant values. Even though the implementation did not reach a particularly deep level, the project was all around very informative and this report will be presenting all of what was requested.

Step 1 – Data Preparation

The dataset I selected from the options that were given the Car Evaluation Dataset. This one was made to address the task of classification. A car's details or "statistics" are given as input to the model and the model is supposed to be able to evaluate the car by placing it in one of the following categorical levels:

- Unacceptable
- Acceptable
- Good
- Very Good

Thus the task was clear. The dataset was read into a pandas data frame. Starting off it was unlabeled so the column headers were added to the data frame. Then the class column was isolated from the rest of the columns, making for the desired output and input data respectively. After that we now have to turn the data into a language a model can understand, numbers. For that the `get_dummies()` function from pandas library was used. Although `get_dummies` is not a sigmoid function like someone would expect. It simply turns categories into binary columns rather than mapping a set of numbers into a noninteger range of 0 - 1. Here would be the moment where we can consider the data ready for training.

Step 2 – Training the model

The project instructions requested 18 different results that we could achieve by tweaking the hidden layer (topology) values, the learning rate values and the learning rate types. Two different topologies were selected:

- (10,)
- (20,10)

The first topology consists of a single hidden layer containing ten neurons. The second topology introduces a deeper architecture with two hidden layers, containing twenty and ten neurons respectively. This allowed a comparison between a simpler and a more complex network structure.

The second experimental parameter was the learning rate. The learning rate controls the size of the adjustments made to the synaptic weights during training. A smaller learning rate causes slower but potentially more stable learning, while a larger learning rate can accelerate training but may overshoot optimal solutions.

Three learning rate values were tested:

- 0.001
- 0.02
- 0.1

The final parameter involved the learning rate schedule used by the optimizer. Scikit-Learn provides three available options:

- Constant
- Invscaling
- Adaptive

We can utilize all of those by wrapping the training part of the code under 3 nested loops one for each variable. Thus 2 Topologies x 3 Learning rate Values x 3 Learning rate types = 18 results on record.

Moving on the train / test split was maintained at the constant 80/20 which made for the better accuracy values comparably. The

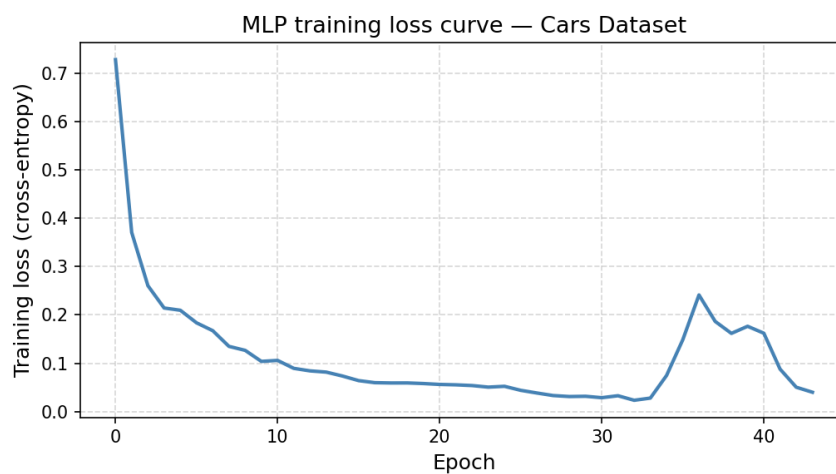
program was constructed in the clearest way possible accounting for the fact that users may want to tweak the topology variables for testing or educational purposes. For that reason all variables have been placed at the top and comments have been used to make the program appear more appealing.

Step 3 – Evaluation and Analysis

It was requested that the results are the error graphs of the models. Thus at the end of every model's training, using the matplotlib library, the corresponding error graphs or loss curves were constructed displayed and saved in the project folder. The attached file SectionA_graphs.zip contains the results of testing the program in my home computer.

Lets take a look at some results and compare them:

Topology: (10,) Learning Rate Value: 0.1

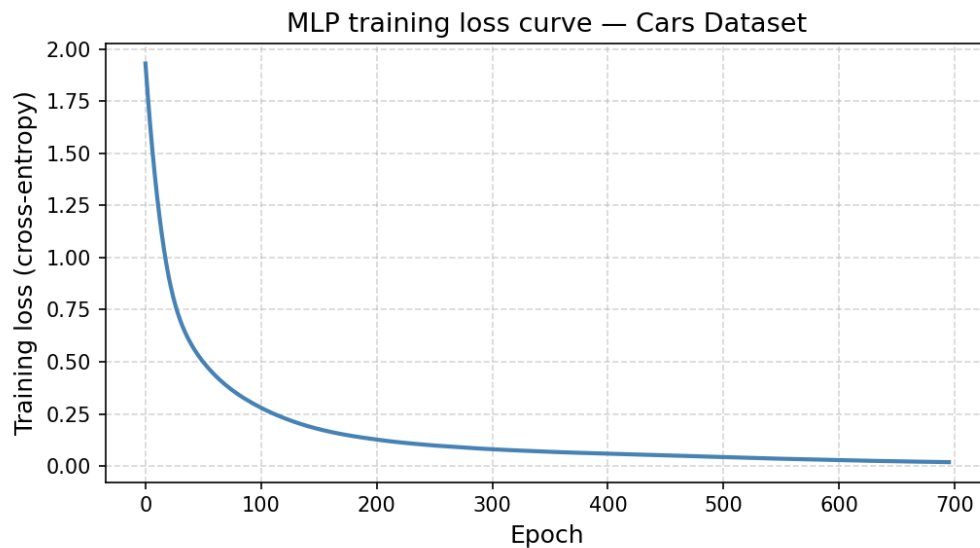


Here we can observe a lack of smoothness. This is due to the learning rate value being high, which essentially means the

learning steps are large, often skipping details. Thus the loss is unstable more than desired lowering accuracy significantly.

To prove that here is the graph corresponding to the same topology with a way smaller learning rate value:

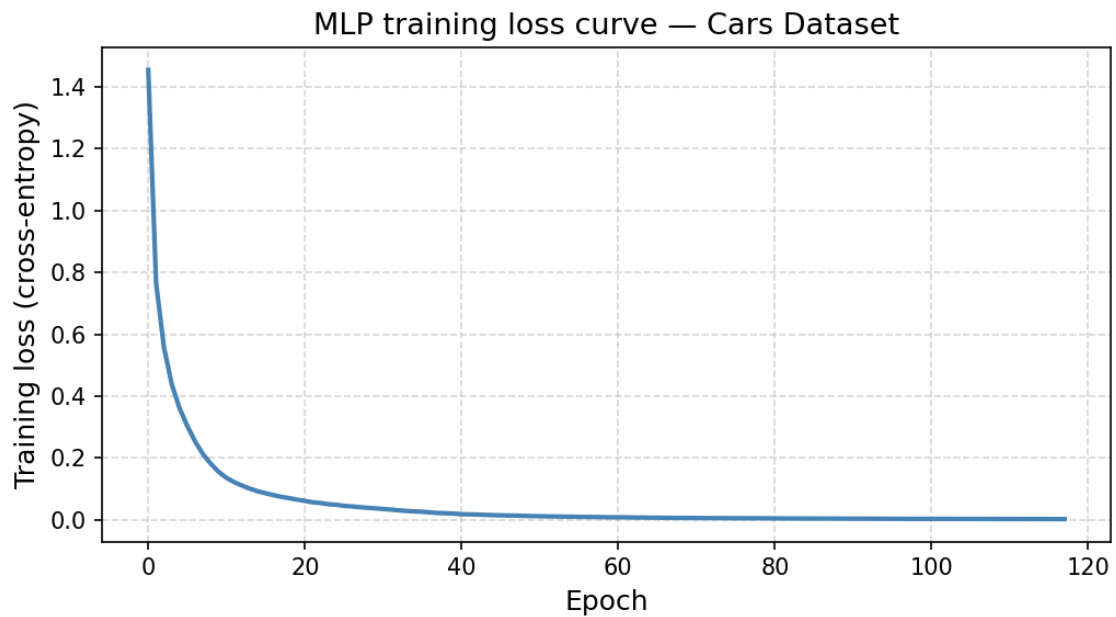
Topology: (10,) Learning Rate Value: 0.001



Now it becomes clear what the problem was earlier. The curve here is extremely smooth which means loss is regulated properly while the epochs increase. Obviously this is more resource intensive, but for models this is essentials and required to reach high accuracy numbers (> 95%). This although is not always the case. The correct set of parameters can minimize loss and regulate it properly. But generally higher learning rate values will mean a less stable loss curve.

Lets move on to our final testing learning rate:

Topology: (10,), Learning Rate Value: 0.02

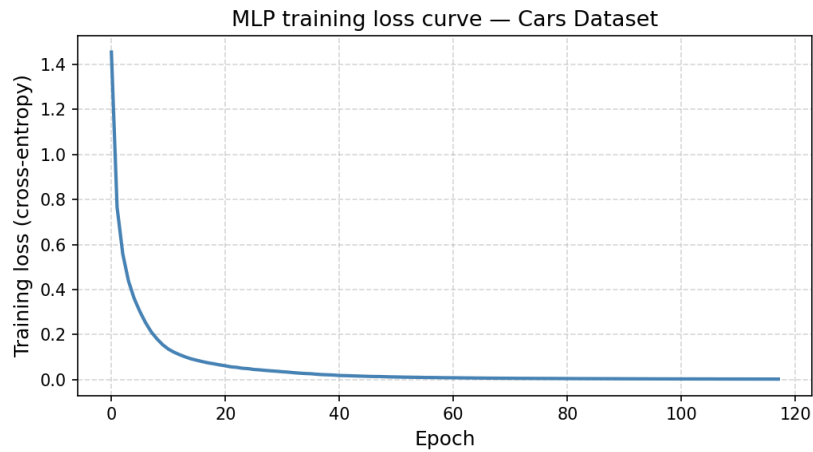


Here we can observe something interesting. With this learning rate loss is not only smoothly regulated, but also quickly minimized. This learning rate seemed to be the optimal value for increasing accuracy in these models.

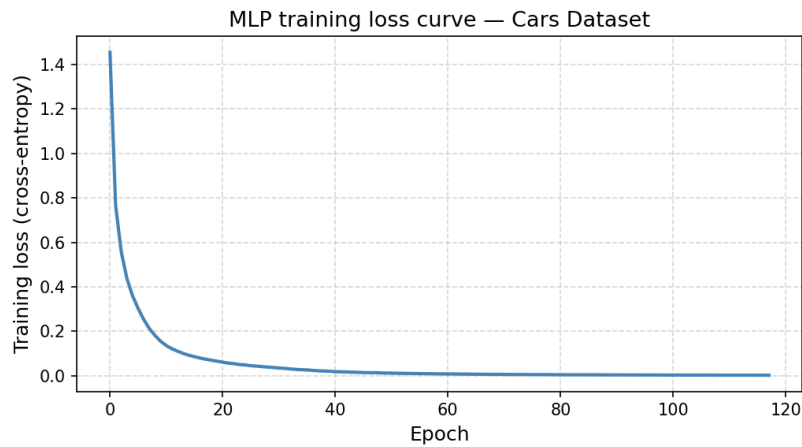
Important Note: The model's loss curves did not seem to be affected much by changing parameters other than the learning rate. This is partly because the cars dataset is fairly simple and a network with 10 neurons and 1 hidden layer which the smallest one used may already have the capacity to respond to this task. Thus a larger network like (20,10) of 20 neurons 11 hidden layers, may have not much extra work to do. The upcoming graphs will be demonstrating the lack of difference between the loss curves assuming the learning rate value stays constant.

For reference the graphs presented utilized the constant learning rate type.

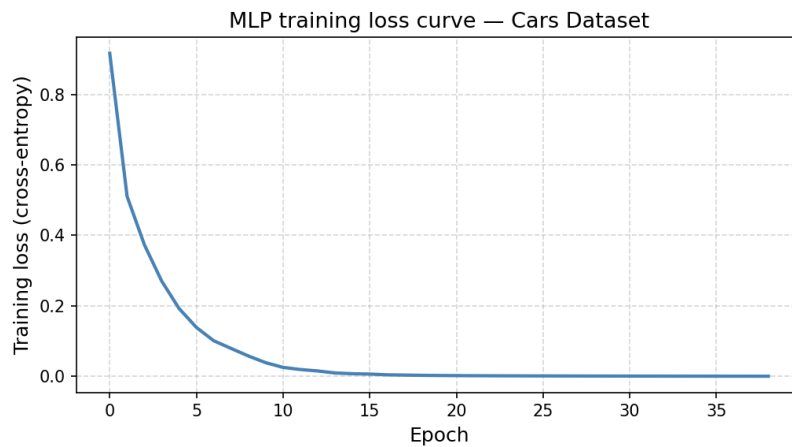
Topology: (10,) Learning Rate: 0.02, Learning Rate Type: Invscaling



Topology: (10,) Learning Rate: 0.02, Learning Rate Type: Adaptive



Topology: (20,10) Learning Rate: 0.02, Learning Rate Type: Invscaling



As it is now clear, there is no significant difference between the 3 error graphs. This means that parameters other than LRV are not significant.

Finally the thing requested was the synaptic weight change graph and error graph of the highest performing model. This was achieved by keeping track of the highest accuracy model in the group and all that model's parameter values while running the nested loops. After we do that we use the saved values to construct the requested plots. The referred to plots will be attached in the SectionA_graphs.zip file along with everything else.

Conclusion

In this project the practical use of an Artificial Neural Network in a real world classification problem was demonstrated. Using the car dataset, a complete machine learning model was implemented and all steps were utilized cleanly. We started by reading, preprocessing, and encoding data continued into formatting and training and finalized by evaluating the produced output.

A total of eighteen different model configurations were tested by varying network topology, learning rate values, and learning rate types. The results showed that the learning rate had the most significant impact on training behavior and model performance. Higher learning rates generally produced less stable loss curves, while appropriately chosen values resulted in smoother convergence and higher classification accuracy. In contrast, changes to the network topology and learning rate type produced relatively minor differences in the particular problem, likely due to the simplicity of the dataset and fact that even the smallest network architecture possessed sufficient capacity to model the underlying patterns.

The project also provided valuable insight into important AI concepts such as one – hot encoding, supervised learning, epochs, loss functions, synaptic weights, and neural network topologies. By generating both loss curves and synaptic weight change graphs, it became possible to visualize the learning process and better understand how neural networks gradually improve their predictions through training.

Overall the objectives of the assignment were met while also providing a strong practical foundation for future study in Artificial Intelligence.

Bibliography

1. GeeksforGeeks. (n.d.). *Derivative of the sigmoid function*. [GeeksforGeeks – Derivative of the sigmoid function](#)
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (n.d.). *Neural network models (supervised learning)*. Scikit-learn. [Scikit-learn Neural Networks Documentation](#)
3. TensorFlow. (n.d.). *Basic classification: Classify images of clothing*. [TensorFlow Keras classification tutorial](#)
4. Keras. (n.d.). *Image classification from scratch*. [Keras image classification example](#)
5. 3Blue1Brown. (2017). *Neural networks* [YouTube playlist]. YouTube. https://www.youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi